

中山大学计算机院本科生实验报告

(2025 学年春季学期)

课程名称：并程序序设计

批改人：

实验	环境配置与串行矩阵乘法	专业（方向）	计算机科学与技术
学号	22320021	姓名	陈安康
Email	1743826979@qq.com	完成日期	2025/3/22

1. 实验目的

配置实验环境。实现串行矩阵乘法并进行优化，分析比较不同优化方案对性能提升的影响

2. 实验过程和核心代码

首先配置好实验环境，我在 Ubuntu20.04 下进行环境配置，安装 Intel MKL 库。运行官方示例代码如下：

```
user@user-virtual-machine:~/并程序序设计$ ./test
This example computes real matrix C=alpha*A*B+beta*C using
Intel(R) MKL function dgemm, where A, B, and C are matrices and
alpha and beta are double precision scalars

Initializing data for matrix multiplication C=A*B for matrix
A(2000x200) and matrix B(200x1000)

Allocating memory for matrices aligned on 64-byte boundary for better
performance

Intializing matrix data

Computing matrix product using Intel(R) MKL dgemm function via CBLAS interface

Computations completed.

Top left corner of matrix A:
  1      2      3      4      5      6
 201    202    203    204    205    206
 401    402    403    404    405    406
 601    602    603    604    605    606
 801    802    803    804    805    806
1001    1002   1003   1004   1005   1006

Top left corner of matrix B:
  -1      -2      -3      -4      -5      -6
-1001   -1002   -1003   -1004   -1005   -1006
-2001   -2002   -2003   -2004   -2005   -2006
-3001   -3002   -3003   -3004   -3005   -3006
-4001   -4002   -4003   -4004   -4005   -4006
-5001   -5002   -5003   -5004   -5005   -5006

Top left corner of matrix C:
-2.6666E+09 -2.6666E+09 -2.6667E+09 -2.6667E+09 -2.6667E+09 -2.6667E+09
-6.6467E+09 -6.6467E+09 -6.6468E+09 -6.6468E+09 -6.6469E+09 -6.647E+09
-1.9627E+10 -1.9627E+10 -1.9627E+10 -1.9627E+10 -1.9627E+10 -1.9627E+10
-1.4607E+10 -1.4607E+10 -1.4607E+10 -1.4607E+10 -1.4607E+10 -1.4607E+10
-1.8587E+10 -1.8587E+10 -1.8587E+10 -1.8587E+10 -1.8588E+10 -1.8588E+10
-2.2567E+10 -2.2567E+10 -2.2567E+10 -2.2567E+10 -2.2568E+10 -2.2568E+10

Deallocating memory
```

实现 python 和 C++版本的串行矩阵乘法。两个矩阵 A 和 B 的规模分别设置为（1024，4096）和（4096，1024）核心代码如下：

C++版本：

```

const int n = 1024, m = 4096, p = 1024;

// 动态分配内存并返回二维数组
void matrixMultiply(double** A, double** B, double** C) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < p; j++) {
            for (int k = 0; k < m; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}

// 初始化二维数组
void initializeMatrix(double** matrix, int rows, int cols) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            matrix[i][j] = (double)rand() / (double)RAND_MAX;
        }
    }
}

int main() {
    srand(static_cast<unsigned int>(time(nullptr))); // 设置随机数种子
    double** A = new double*[n];
    for (int i = 0; i < n; i++) {
        A[i] = new double[m];
    }
    double** B = new double*[m];
    for (int i = 0; i < m; i++) {
        B[i] = new double[p];
    }
    double** C = new double*[n];
    for (int i = 0; i < n; i++) {
        C[i] = new double[p];
        for (int j = 0; j < p; j++) {
            C[i][j] = 0.0;
        }
    }
    initializeMatrix(A, n, m);
    initializeMatrix(B, m, p);

    auto start = high_resolution_clock::now();
    matrixMultiply(A, B, C);
    auto end = high_resolution_clock::now();

    duration<double> elapsed = end - start;
    cout << "Execution Time: " << elapsed.count() << " sec" << endl;
}

```

python 版本:

```

import numpy as np
import time

def matrix_multiply(A, B):
    n, m = len(A), len(A[0])
    p = len(B[0])
    C = [[0] * p for _ in range(n)]

    for i in range(n):
        for j in range(p):
            for k in range(m):
                C[i][j] += A[i][k] * B[k][j]

    return C

n = 1024
m = 4096
p = 1024
# 生成 500x500 矩阵
A = [[1] * n for _ in range(m)]
B = [[1] * m for _ in range(p)]

start_time = time.time()
C = matrix_multiply(A, B)
end_time = time.time()

print(f"Execution Time: {end_time - start_time} sec")

```

两者运行结果对比如下:

cpp:

```
user@user-virtual-machine:~/并行程序设计/hw1$ g++ matrix_cpp.cpp -o matrix_cpp
user@user-virtual-machine:~/并行程序设计/hw1$ ./matrix_cpp
Execution Time: 88.3065 sec
```

python:

```
[Running] python -u "/home/user/并行程序设计/hw1/matrix_py.py"
Execution Time: 2230.818442583084 sec

[Done] exited with code=0 in 2231.97 seconds
```

调整循环顺序:

合适的循环顺序调整可以提高缓存命中率, 在 cpp 版的程序基础上, 调整循环顺序如下:

```
// 动态分配内存并返回二维数组
void matrixMultiply(double** A, double** B, double** C) {
    for (int i = 0; i < n; i++) {
        for (int k = 0; k < m; k++) {
            for (int j = 0; j < p; j++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}
```

运行结果如下:

```
user@user-virtual-machine:~/并行程序设计/hw1$ g++ matrix_cpp_change.cpp -o matrix_cpp_change
user@user-virtual-machine:~/并行程序设计/hw1$ ./matrix_cpp_change
Execution Time: 16.7962 sec
```

对比未调整循环顺序的缓存命中率:

未调整循环顺序:

```
==6696== I   refs:      26,238,685,551
==6696== Il  misses:      1,622
==6696== Lli misses:      1,564
==6696== Il  miss rate:    0.00%
==6696== Lli miss rate:    0.00%
==6696==
==6696== D   refs:      11,536,074,429 (10,956,776,485 rd + 579,297,944 wr)
==6696== D1  misses:      639,697,293 ( 638,507,538 rd + 1,189,755 wr)
==6696== Lld misses:      508,971,224 ( 507,782,552 rd + 1,188,672 wr)
==6696== D1  miss rate:    5.5% ( 5.8% + 0.2% )
==6696== Lld miss rate:    4.4% ( 4.6% + 0.2% )
==6696==
==6696== LL refs:      639,698,915 ( 638,509,160 rd + 1,189,755 wr)
==6696== LL misses:      508,972,788 ( 507,784,116 rd + 1,188,672 wr)
==6696== LL miss rate:    1.3% ( 1.4% + 0.2% )
```

调整循环顺序:

```

==7435== I   refs:      75,106,570,754
==7435== Il  misses:      1,622
==7435== Lli misses:      1,564
==7435== Il  miss rate:      0.00%
==7435== Lli miss rate:      0.00%
==7435==
==7435== D   refs:      33,039,166,025 (31,481,367,889 rd + 1,557,798,136 wr)
==7435== D1  misses:      187,953,513 ( 186,763,767 rd + 1,189,746 wr)
==7435== Lld misses:      187,946,752 ( 186,758,081 rd + 1,188,671 wr)
==7435== D1  miss rate:      0.6% (      0.6% +      0.1% )
==7435== Lld miss rate:      0.6% (      0.6% +      0.1% )
==7435==
==7435== LL refs:      187,955,135 ( 186,765,389 rd + 1,189,746 wr)
==7435== LL misses:      187,948,316 ( 186,759,645 rd + 1,188,671 wr)
==7435== LL miss rate:      0.2% (      0.2% +      0.1% )

```

编译优化：

使用`__restrict`关键字告诉编译器指针没有重叠，使编译器利用这一信息提高内存访问效率，帮助编译器生成更优的代码。设置最高优化等级（`-O3`），针对本地机器生成最优代码（`-march=native`）。

```

// 采用 `__restrict` 提示编译器进行优化
void matrixMultiply(const double* __restrict A,
                   const double* __restrict B,
                   double* __restrict C, int n, int m, int p)
{
    for (int i = 0; i < n; i++) {
        for (int k = 0; k < m; k++) {
            for (int j = 0; j < p; j++) {
                C[i * p + j] += A[i * m + k] * B[k * p + j];
            }
        }
    }
}

```

编译命令：

```
g++ -O3 -march=native -ffast-math matrix_cpp_compile.cpp -o matrix_cpp_compile
```

运行结果：

```

user@user-virtual-machine:~/并行程序设计/hw1$ g++ -O3 -march=native -ffast-math matrix_cpp_compile.cpp -o matrix_cpp_compile
user@user-virtual-machine:~/并行程序设计/hw1$ ./matrix_cpp_compile
Execution Time: 2.2424 sec

```

循环展开：

将串行算法进行循环展开：


```

// 采用 `__restrict` + 循环展开 (Unrolling)
void matrixMultiply(const double* __restrict__ A,
                   const double* __restrict__ B,
                   double* __restrict__ C, int n, int m, int p)
{
    // 初始化 C 矩阵, 避免使用未定义值
    // for (int i = 0; i < n * p; i++) {
    //     C[i] = 0.0;
    // }

    // 进行矩阵乘法计算, 并使用 4x展开加速
    for (int i = 0; i < n; i++) {
        for (int k = 0; k < m; k++) {
            double Aik = A[i * m + k]; // 预取 A[i][k], 减少内存访问

            for (int j = 0; j < p; j += 2) { // 2 次展开, 展开次数越多, 效果反而越不好
                C[i * p + j] += Aik * B[k * p + j];
                C[i * p + j + 1] += Aik * B[k * p + j + 1];
                // C[i * p + j + 2] += Aik * B[k * p + j + 2];
                // C[i * p + j + 3] += Aik * B[k * p + j + 3];
            }
        }
    }
}

```

运行效果如下:

```

user@user-virtual-machine:~/并行程序设计/hw1$ g++ -O3 -march=native -ffast-math matrix_cpp_loop.cpp -o matrix_cpp_loop
user@user-virtual-machine:~/并行程序设计/hw1$ ./matrix_cpp_loop
Execution Time: 3.34851 sec

```

对比普通编译优化的运行效果, 发现效果反而下降了, 并且展开越大, 效果越不好, 经过查找, 发现可能是手动循环展开可能与其他优化发生了冲突, 导致效率不降反升。

简单修改代码, 对比不使用编译优化时的循环展开相比于同样不使用编译优化的程序有性能上的提升。

```

void matrixMultiply(const double* A,
                   const double* B,
                   double* C, int n, int m, int p)
{
    // 初始化 C 矩阵, 避免使用未定义值
    // for (int i = 0; i < n * p; i++) {
    //     C[i] = 0.0;
    // }

    // 进行矩阵乘法计算, 并使用 4x展开加速
    for (int i = 0; i < n; i++) {
        for (int k = 0; k < m; k++) {
            double Aik = A[i * m + k]; // 预取 A[i][k], 减少内存访问

            for (int j = 0; j < p; j += 8) { // 8 次展开
                C[i * p + j] += Aik * B[k * p + j];
                C[i * p + j + 1] += Aik * B[k * p + j + 1];
                C[i * p + j + 2] += Aik * B[k * p + j + 2];
                C[i * p + j + 3] += Aik * B[k * p + j + 3];
                C[i * p + j + 4] += Aik * B[k * p + j + 4];
                C[i * p + j + 5] += Aik * B[k * p + j + 5];
                C[i * p + j + 6] += Aik * B[k * p + j + 6];
                C[i * p + j + 7] += Aik * B[k * p + j + 7];
            }
        }
    }
}

```

```

user@user-virtual-machine:~/并行程序设计/hw1$ g++ matrix_cpp_loop.cpp -o matrix_cpp_loop
user@user-virtual-machine:~/并行程序设计/hw1$ ./matrix_cpp_loop
Execution Time: 12.3763 sec
user@user-virtual-machine:~/并行程序设计/hw1$ ./matrix_cpp_loop
Execution Time: 11.9997 sec
user@user-virtual-machine:~/并行程序设计/hw1$ ./matrix_cpp_loop
Execution Time: 12.0608 sec
user@user-virtual-machine:~/并行程序设计/hw1$ ./matrix_cpp_loop
Execution Time: 11.9464 sec

```

可以发现, 在不启用编译优化时, 手动的循环展开相比于普通的程序可以实现性能提升。

Strassen 算法:

采用 Strassen 算法进行优化, 虽然 Strassen 算法常常被用来应用于方阵场景, 但由于我的非方阵规模都是 2 的整数倍, 因此可以正常应用。关键代码如下:

```

// Strassen 递归算法
void strassen(double** A, double** B, double** C, int n, int m, int p) {
    if (n <= 64 || m <= 64 || p <= 64) {
        for (int i = 0; i < n; i++)
            for (int k = 0; k < m; k++)
                for (int j = 0; j < p; j++) {
                    C[i][j] += A[i][k] * B[k][j];
                    // flop_count += 2;
                }
        return;
    }

    int new_n = n / 2, new_m = m / 2, new_p = p / 2;

    double** A11 = allocateMatrix(new_n, new_m);
    double** A12 = allocateMatrix(new_n, new_m);
    double** A21 = allocateMatrix(new_n, new_m);
    double** A22 = allocateMatrix(new_n, new_m);

    double** B11 = allocateMatrix(new_m, new_p);
    double** B12 = allocateMatrix(new_m, new_p);
    double** B21 = allocateMatrix(new_m, new_p);
    double** B22 = allocateMatrix(new_m, new_p);

    double** C11 = allocateMatrix(new_n, new_p);
    double** C12 = allocateMatrix(new_n, new_p);
    double** C21 = allocateMatrix(new_n, new_p);
    double** C22 = allocateMatrix(new_n, new_p);

    double** M1 = allocateMatrix(new_n, new_p);
    double** M2 = allocateMatrix(new_n, new_p);
    double** M3 = allocateMatrix(new_n, new_p);
    double** M4 = allocateMatrix(new_n, new_p);
    double** M5 = allocateMatrix(new_n, new_p);
    double** M6 = allocateMatrix(new_n, new_p);
    double** M7 = allocateMatrix(new_n, new_p);

```

```

double** tempA = allocateMatrix(new_n, new_m);
double** tempB = allocateMatrix(new_m, new_p);

for (int i = 0; i < new_n; i++) {
    for (int j = 0; j < new_p; j++) {
        A11[i][j] = A[i][j];
        A12[i][j] = A[i][j + new_m];
        A21[i][j] = A[i + new_n][j];
        A22[i][j] = A[i + new_n][j + new_m];

        B11[i][j] = B[i][j];
        B12[i][j] = B[i][j + new_p];
        B21[i][j] = B[i + new_m][j];
        B22[i][j] = B[i + new_m][j + new_p];
    }
}

addMatrix(A11, A22, tempA, new_n, new_m);
addMatrix(B11, B22, tempB, new_m, new_p);
strassen(tempA, tempB, M1, new_n, new_m, new_p);

addMatrix(A21, A22, tempA, new_n, new_m);
strassen(tempA, B11, M2, new_n, new_m, new_p);

subMatrix(B12, B22, tempB, new_m, new_p);
strassen(A11, tempB, M3, new_n, new_m, new_p);

subMatrix(B21, B11, tempB, new_m, new_p);
strassen(A22, tempB, M4, new_n, new_m, new_p);

addMatrix(A11, A12, tempA, new_n, new_m);
strassen(tempA, B22, M5, new_n, new_m, new_p);

subMatrix(A21, A11, tempA, new_n, new_m);
addMatrix(B11, B12, tempB, new_m, new_p);
strassen(tempA, tempB, M6, new_n, new_m, new_p);

```



```

subMatrix(A12, A22, tempA, new_n, new_m);
addMatrix(B21, B22, tempB, new_m, new_p);
strassen(tempA, tempB, M7, new_n, new_m, new_p);

addMatrix(M1, M4, tempA, new_n, new_p);
subMatrix(tempA, M5, tempB, new_n, new_p);
addMatrix(tempB, M7, C11, new_n, new_p);

addMatrix(M3, M5, C12, new_n, new_p);
addMatrix(M2, M4, C21, new_n, new_p);

addMatrix(M1, M3, tempA, new_n, new_p);
subMatrix(tempA, M2, tempB, new_n, new_p);
addMatrix(tempB, M6, C22, new_n, new_p);

// **合并结果**
for (int i = 0; i < new_n; i++) {
    for (int j = 0; j < new_p; j++) {
        C[i][j] = C11[i][j];
        C[i][j + new_p] = C12[i][j];
        C[i + new_n][j] = C21[i][j];
        C[i + new_n][j + new_p] = C22[i][j];
    }
}

freeMatrix(A11, new_n); freeMatrix(A12, new_n);
freeMatrix(A21, new_n); freeMatrix(A22, new_n);
freeMatrix(B11, new_m); freeMatrix(B12, new_m);
freeMatrix(B21, new_m); freeMatrix(B22, new_m);
freeMatrix(C11, new_n); freeMatrix(C12, new_n);
freeMatrix(C21, new_n); freeMatrix(C22, new_n);
freeMatrix(M1, new_n); freeMatrix(M2, new_n);
freeMatrix(M3, new_n); freeMatrix(M4, new_n);
freeMatrix(M5, new_n); freeMatrix(M6, new_n);
freeMatrix(M7, new_n);
freeMatrix(tempA, new_n); freeMatrix(tempB, new_m);
}

```

运行效果如下：

```

user@user-virtual-machine:~/并行程序设计/hw1$ g++ matrix_cpp_strassen.cpp -o matrix_cpp_strassen
user@user-virtual-machine:~/并行程序设计/hw1$ ./matrix_cpp_strassen
Computation finished!
Execution Time: 10.4194 sec

```

编译优化后运行结果：

```

user@user-virtual-machine:~/并行程序设计/hw1$ g++ -O3 -march=native -ffast-math matrix_cpp_strassen.cpp -o
matrix_cpp_strassen
user@user-virtual-machine:~/并行程序设计/hw1$ ./matrix_cpp_strassen
Computation finished!
Execution Time: 1.24721 sec

```

可以看到，Strassen 算法大幅度提升了效率。

Intel MKL 优化串行矩阵乘法：

Intel MKL (Intel Math Kernel Library, 英特尔数学核心库) 是 Intel 提供的高性能数学运算库, 专门针对 Intel 处理器进行了深度优化, 广泛用于 科学计算、机器学习、并行计算和高性能计算 (HPC) 领域。

使用 MKL 库进行优化, 关键代码如下:

```
int main() {
    srand(static_cast<unsigned int>(time(nullptr))); // 设置随机数种子

    // 采用 MKL 提供的 mkl_malloc 进行对齐分配
    double *A = (double*)mkl_malloc(n * m * sizeof(double), 64);
    double *B = (double*)mkl_malloc(m * p * sizeof(double), 64);
    double *C = (double*)mkl_malloc(n * p * sizeof(double), 64);

    if (A == nullptr || B == nullptr || C == nullptr) {
        cout << "Memory allocation failed!" << endl;
        return -1;
    }

    initializeMatrix(A, n, m);
    initializeMatrix(B, m, p);
    fill_n(C, n * p, 0.0); // 初始化 C 为 0

    auto start = high_resolution_clock::now();

    // 采用 Intel MKL 进行矩阵乘法计算
    cblas_dgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
                n, p, m, 1.0, A, m, B, p, 0.0, C, p);

    auto end = high_resolution_clock::now();
    duration<double> elapsed = end - start;

    cout << "Execution Time with Intel MKL: " << elapsed.count() << " sec" << endl;

    // 释放内存
    mkl_free(A);
    mkl_free(B);
    mkl_free(C);

    return 0;
}
```

运行结果如下:

```
user@user-virtual-machine:~/并行程序设计/hw1$ g++ matrix_cpp_mkl.cpp -I/usr/include/mkl -L/usr/lib/x86-64-linux-gnu -lmkl_rt -o matrix_cpp_mkl
user@user-virtual-machine:~/并行程序设计/hw1$ ./matrix_cpp_mkl
Execution Time with Intel MKL: 0.230603 sec
```

可以看到 MKL 库优化下程序效率大幅度上升。

3. 实验结果

由于是非方阵, 浮点数性能公式如下:

$$\text{GFLOPS} = \frac{2 * N * M * P}{T * 10^9}$$

由于 Strassen 算法的浮点数操作计算公式过于复杂，这里在代码中进行手动记录，添加一个数据进行实时记录，统计得到结果如下：

```
Execution Time: 12.6127 sec
Total Floating Point Operations: 5035269152
[1] + Done "/usr/bin/gd
```

计算浮点数峰值性能估计，我的虚拟机支持 FMA 和 avx2：

```
标记：
fpu vme de pse tsc msr pae mce cx8 apic se
p mtrr pge mca cmov pat pse36 clflush mmx
fxsr sse sse2 ss syscall nx pdpe1gb rdtscp
lm constant_tsc arch_perfmon rep_good nop
l xtopology tsc_reliable nonstop_tsc cpuid
tsc_known_freq pni pclmulqdq ssse3 fma cx
16 sse4_1 sse4_2 x2apic movbe popcnt tsc_d
eadline_timer aes xsave avx f16c rdrand hy
pervisor lahf_lm abm 3dnowprefetch ssbd ib
rs ibpb stibp ibrs_enhanced fsgsbase tsc_a
djust bmi1 avx2 smep bmi2 erms invpcid rds
eed adx smap clflushopt clwb sha_ni xsaveo
pt xsavec xgetbv1 xsaves avx_vnni arat umi
p pku ospke gfni vaes vpclmulqdq rdpid mov
diri movdir64b fsrm md_clear serialize flu
sh_lid arch_capabilities
```

Peak = 时钟频率 * 处理器数目 * 处理器核心数 * (8 * 2)

我的虚拟机配置如下：

```
user@user-virtual-machine:~$ lscpu | grep 'Model name\|CPU MHz\|CPU max MHz\|FLO
PS'
CPU MHz: 2688.001
```

设备	摘要	处理器
内存	4 GB	处理器数量(P): 2
处理器	2	每个处理器的内核数量(C): 1
硬盘 (SCSI)	200 GB	处理器内核总数: 2
CD/DVD (SATA)	正在使用文件 D:\Virtual Mac...	
网络适配器	自定义 (VMnet8 (NAT))	

计算如下：

Peak = 2.688 * 2 * 1 * (8 * 2) = 86.016 GFLOPS

表格填入如下，使用编译优化后的数据进行填入：

版本	实现描述	运行时间 (sec)	相对加速度 比(相对于前 一个版本)	绝对加速度 比(相对于 python 版本)	浮点性能 (GFLOPS)	峰值性能百 分比%
1	Python	2230.8184	-	1	0.00385	0.004476
2	C/C++	88.3065	25.26	25.26	0.09727	0.11
3	调整循环顺序	16.7962	5.25	132.81	0.51142	0.59
4	编译优化	2.2424	7.49	994.8351	3.83068	4.53
5	循环展开	3.3485	0.67	666.21	2.56530	2.98
6	Strassen 算法 优化	1.2472	(对比编译 优化) 1.78	1788.66	4.03725	4.69
7	Intel MKL	0.2306	5.41	9675.66	37.25036	43.30

对比可以发现，C/C++的性能比 python 高出不少。合适的循环顺序调整可以大幅度提高缓存命中率，从而大幅度提高性能。采用合适编译选项、优化算法或者高性能库可以更进一步提升程序性能。不过串行方面再怎么优化，整个 CPU 的性能依旧没有被完全压榨，仍然有等待开发的空间。因此需要并行化来进一步提高 CPU 利用率。

4. 实验感想

在实现 Strassen 算法时，由于此算法是用来解决方阵的，问大模型说非方阵实现的话需要填补成方阵，可那样肯定会降低性能，由于我的矩阵规模是 2 的倍数，因此合适直接执行 Strassen 算法，调整划分的大小后顺利执行。

在统计 Strassen 浮点数操作次数时，由于递归计算确切的次数十分复杂，采用在运算中实时统计的方法获取操作次数。

通过此次实验切实体会到 python 和 C 的性能差异，以及各种优化方法的性能提升。意识到串行优化存在一定的瓶颈，需要并行化来进一步压榨 cpu 性能。